

UNIVERSITAS CAKRAWALA

LAPORAN INDIVIDU

UJIAN TENGAH SEMESTER (UTS)

Manajemen Resep (Add, Remove, Change)

Cookbook System - Aplikasi Manajemen Resep Berbasis Python

Nama	Lars Alfarabi Gunawan
NIM	24110300067
Mata Kuliah	Programming Fundamental
Program Studi	Ilmu Komputer
Dosen Pengampu	Fikri Hanif, S.T., M.S.M.
Semester / Kelas	Ganjil / Prog4
Tahun Akademik	2026/2027

Oktober 2026

1. Kontribusi dan Analisis IPO

Bagian yang saya tangani di Cookbook System adalah manajemen resep, yaitu seluruh berkas controls.py beserta fungsi rebuild_data() yang menyusun ulang struktur pencarian. Tiga fungsi utamanya adalah add_recipe(), remove_recipe(), dan change_recipe(). Sebagai ketua kelompok, saya juga yang mengatur jadwal penggabungan kode supaya keempat bagian tidak saling menimpa saat digabung.

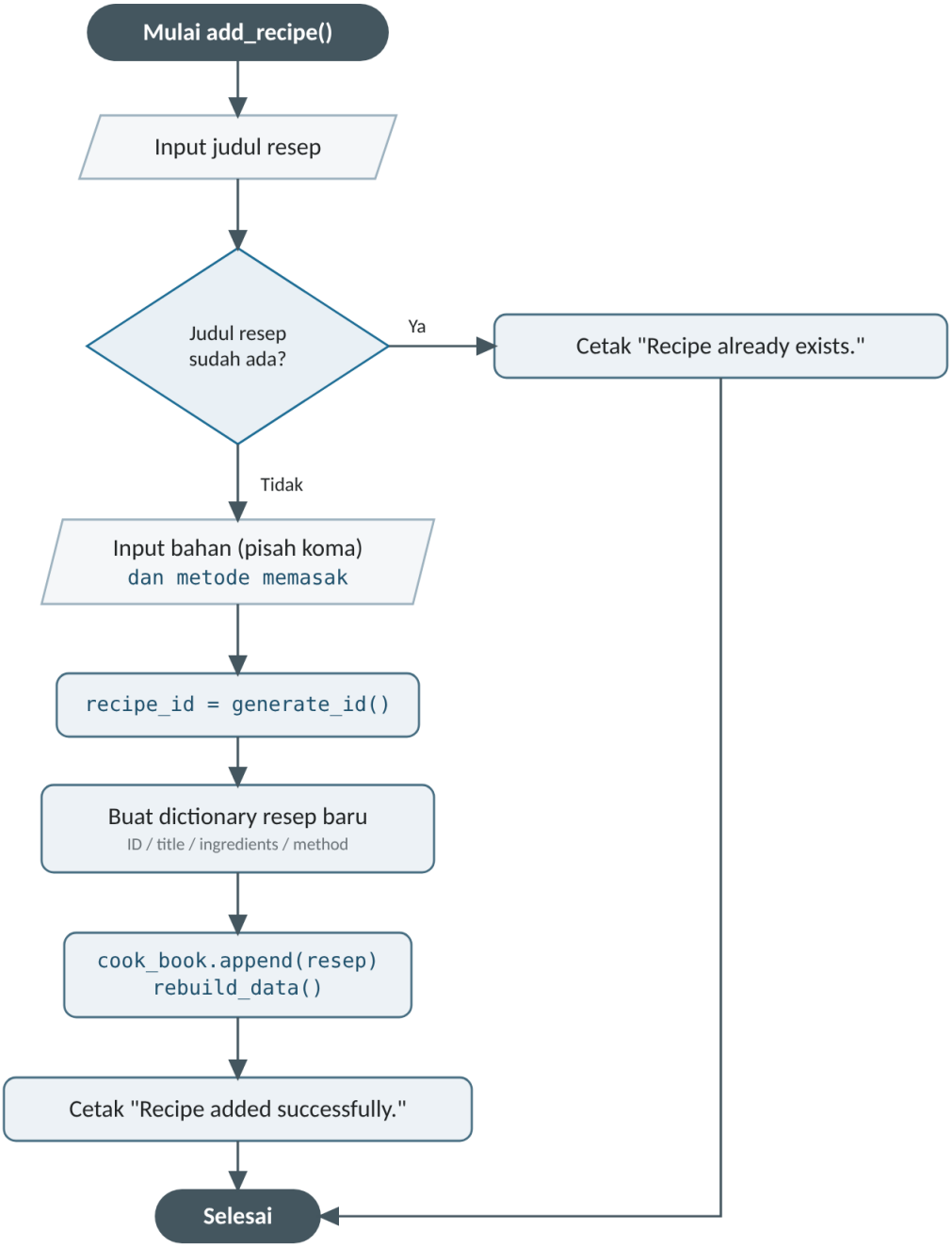
Tanpa menu tambah, hapus, dan ubah, cook_book hanya akan menjadi daftar resep beku yang tidak bisa dikoreksi. Salah ketik judul tidak bisa diperbaiki, resep yang tidak lagi diminati tidak bisa dibuang, resep baru dari teman tidak punya tempat masuk. Lewat tiga menu ini, isi koleksi akhirnya mengikuti kebutuhan pemakainya.

Bagian	Rincian
Input	Judul resep (str), bahan dipisah koma (str), metode memasak (str), dan nomor resep (int hasil int(input())) untuk fitur hapus dan ubah.
Process	Pemeriksaan judul ganda lewat perulangan for, pembuatan ID acak lewat generate_id(), penyusunan dictionary resep, penambahan atau penghapusan dari list cook_book, lalu rebuild_data() menyusun ulang hash table dan trie.
Output	Pesan Recipe added/removed/changed successfully., pesan penolakan Recipe already exists. atau Recipe not found., serta tampilnya daftar resep terbaru ketika pengguna memilih opsi 5.

Tabel 1. Analisis IPO fitur manajemen resep

2. Flowchart Fitur

Dari tiga fungsi yang saya pegang, add_recipe() punya alur paling lengkap karena di dalamnya ada pemeriksaan kondisi, pemanggilan dua fungsi milik anggota lain, dan penyusunan ulang struktur data. Alurnya saya gambarkan pada Gambar 1. Cabang kiri adalah jalur penolakan: begitu judul ternyata sudah dipakai, fungsi langsung mencetak pesan dan berhenti tanpa menyentuh data sama sekali. Cabang bawah adalah jalur sukses yang berakhir pada pemanggilan rebuild_data(), bagian yang menurut saya paling penting agar hash table dan trie tidak menyimpan resep yang sudah tidak ada.



Gambar 1. Flowchart fungsi add_recipe() pada controls.py

3. Elemen Pemrograman Dasar

Variabel di fitur saya semuanya bersifat lokal, dibuat baru setiap kali fungsi dipanggil, dan tipe datanya mengikuti isi yang dimasukkan pengguna. Rinciannya saya susun di Tabel 2. Satu hal yang menarik ada pada ingredients: hasil input() berupa string, lalu berubah menjadi list begitu diproses split(","). Jadi satu variabel yang sama bisa berganti bentuk, dan di sini pemahaman tipe data benar-benar terpakai.

Variabel	Tipe	Isi / Contoh
title	str	'Fried banana'
ingredients	str lalu list	'banana, flour, oil' menjadi ['banana', ' flour', ' oil'] setelah split(",")
cooking_method	str	'Fried'
recipe_id	int	hasil generate_id(), misal 226
recipe	dict	{'ID': 226, 'title': 'Fried banana', ...}
recipe_number	int	14, hasil int(input())

Tabel 2. Variabel pada fitur manajemen resep

Masukan pengguna diambil dengan input() yang selalu menghasilkan string, sehingga nomor resep harus saya konversi lebih dulu dengan int() sebelum dipakai. Keluaran ditangani print() berupa pesan status, sementara tampilan resepnya sendiri dikerjakan show_recipe() milik Dhafi. Untuk operator, fitur saya memakai operator perbandingan == ketika membandingkan judul lama dan judul baru, keduanya setelah diubah ke huruf kecil supaya Fried Banana dan fried banana dianggap sama. Operator logika or dan method remove() juga terlibat di jalur hapus resep.

Kutipan file controls.py: operator == dan .lower() pada pemeriksaan judul ganda

```
title = input("Enter recipe name: ")

for recipe in data.cook_book:
    if recipe["title"].lower() == title.lower():
        print("Recipe already exists.")
        return
```

4. Percabangan dan Perulangan

Ada tiga pola percabangan di fitur saya dan semuanya berbentuk pemeriksaan di awal fungsi. Pertama, if judul sudah ada di cook_book akan mencetak pesan penolakan lalu return, jadi sisa kode tidak dijalankan. Kedua dan ketiga ada di remove_recipe() dan change_recipe(): jika get_recipe_by_number() mengembalikan None berarti nomor tidak valid, fungsi langsung mencetak Recipe not found. dan keluar. Dengan pola begitu, pengubahan data hanya terjadi setelah semua pemeriksaan lolos.

Untuk perulangan, saya memakai for dan while pada tempat yang berbeda dan itu memang sesuai karakternya. Perulangan for saya pakai untuk memeriksa seluruh isi cook_book saat mencari judul ganda karena jumlahnya sudah pasti, yaitu sebanyak resep yang tersimpan. Perulangan while saya warisi dari menu utama dan submenu milik Dhafi yang berjalan terus sampai pengguna memilih keluar, karena jumlah putarannya memang tidak bisa diprediksi. Fungsi generate_id() milik Alexander juga memakai while True dan berhenti begitu ID acak yang belum terpakai ditemukan, contoh perulangan yang berhentinya ditentukan oleh kondisi data.

5. Fungsi dan Struktur Data

Fungsi yang mau saya bahas adalah add_recipe(). Fungsi ini tidak punya parameter dan tidak memakai return secara eksplisit, jadi nilai baliknya adalah None. Awalnya itu terasa aneh bagi saya, tetapi alasannya masuk akal: seluruh datanya justru dibaca langsung dari input() pengguna dan bekerja pada modul data yang diimpor, jadi tidak ada yang perlu dikirim lewat parameter. Hasil kerjanya terlihat lewat perubahan isi data.cook_book, bukan lewat nilai yang dikembalikan. Di dalamnya fungsi ini memanggil dua fungsi anggota lain, generate_id() dan rebuild_data(), sehingga add_recipe() sekaligus jadi titik temu tiga kontribusi anggota kelompok.

Kutipan file controls.py: fungsi add_recipe()

```
def add_recipe():
    print("\n===== ADD RECIPE =====")

    title = input("Enter recipe name: ")

    for recipe in data.cook_book:
        if recipe["title"].lower() == title.lower():
            print("Recipe already exists.")
            return

    ingredients = input("Enter ingredients separated by comma: ")
    ingredients = ingredients.split(",")

    cooking_method = input("Enter cooking method: ")

    recipe_id = data.generate_id()

    recipe = {
        "ID": recipe_id,
        "title": title,
        "ingredients": ingredients,
        "cooking_method": cooking_method
    }

    data.cook_book.append(recipe)
    data.rebuild_data()

    print("Recipe added successfully.")
```

List dan dictionary di aplikasi ini saling menopang. List cook_book menjadi wadah seluruh resep dan saya memakai method append() untuk menambah serta remove() untuk menghapus. Dictionary dipakai

dua lapis: setiap resep adalah dictionary dengan kunci ID, title, ingredients, dan cooking_method, lalu struktur milik Ikmal dan Dhafi membangun pencarian di atasnya. Jadi setiap kali fungsi saya mengubah cook_book, dua struktur turunan itu harus disusun ulang, dan itulah tugas rebuild_data() yang saya panggil di akhir setiap operasi.

6. Penelusuran Kode dan Pengujian

Skenario yang saya telusuri adalah menambah resep Fried banana. Jalannya tercatat langkah demi langkah di Tabel 3. Perhatikan bahwa ID yang tercantum adalah contoh hasil nyata dari pengujian unit; setiap kali program dijalankan ulang, angkanya bisa berbeda karena sifatnya acak, tetapi pola penelusurannya selalu sama.

Langkah	Kode yang Dijalankan	Keadaan / Hasil
1	title = input("Enter recipe name: ")	title = "Fried banana"
2	for recipe in data.cook_book:	14 resep diperiksa satu per satu, tidak ada judul yang sama
3	ingredients = ingredients.split(",")	ingredients = ['banana', ' flour', ' oil', ' sugar']
4	cooking_method = input(...)	cooking_method = "Fried"
5	recipe_id = data.generate_id()	hasil uji unit mengembalikan 226, belum pernah dipakai
6	recipe = {"ID": recipe_id, ...}	dictionary resep lengkap terbentuk
7	data.cook_book.append(recipe)	cook_book kini berisi 15 resep
8	data.rebuild_data()	hash table dan trie disusun ulang dari 15 resep
9	print("Recipe added successfully.")	pesan sukses tampil, kembali ke menu

Tabel 3. Penelusuran add_recipe() untuk resep Fried banana

Pengujian fitur saya lakukan dalam tiga skenario berikut, semuanya dijalankan dari kondisi awal 14 resep. Skenario pertama membuktikan resep benar-benar masuk, skenario kedua membuktikan penolakan judul ganda, dan skenario ketiga menguji jalur ubah yang memakai nomor resep.

Skenario uji 1: menambah resep baru lalu memverifikasi lewat opsi 5 (dipadatkan)

```
===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 1

===== ADD RECIPE =====
```

```
Enter recipe name: Fried banana
Enter ingredients separated by comma: banana, flour, oil, sugar
Enter cooking method: Fried
Recipe added successfully.

===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 5

===== ALL RECIPES =====

-------------------------
Recipe Number: 1
Title: Boiled egg
Ingredients: ['egg', 'water']
Cooking method: Boiled
-------------------------

-------------------------
Recipe Number: 2
Title: Fried egg
Ingredients: ['egg', 'oil']
Cooking method: Fried
-------------------------

     ... resep nomor 3 sampai 14 tampil dengan format yang sama ...

Recipe Number: 14
Title: Chicken soup
Ingredients: ['chicken', 'curry', 'coconut milk']
Cooking method: Simmered
-------------------------
-------------------------
Recipe Number: 15
Title: Fried banana
Ingredients: ['banana', ' flour', ' oil', ' sugar']
Cooking method: Fried
-------------------------

===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 6
Goodbye.
```

Skenario uji 2: penambahan judul yang sudah ada ditolak

```
===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 1
```

```
===== ADD RECIPE =====
Enter recipe name: Pancakes
Recipe already exists.

===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 6
Goodbye.
```

Skenario uji 3: mengubah resep nomor 14 lalu melihat hasilnya (dipadatkan)

```
===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 3

===== CHANGE RECIPE =====
Enter recipe number: 14

Current recipe:
Title: Chicken soup
Ingredients: ['chicken', 'curry', 'coconut milk']
Cooking method: Simmered


     ... resep nomor 1 sampai 13 tampil dengan format yang sama ...

Cooking method: Fried
-------------------------

-------------------------
Recipe Number: 13
Title: Baked fish
Ingredients: ['fish', 'lemon', 'salt']
Cooking method: Baked
-------------------------

-------------------------
Recipe Number: 14
Title: Chicken coconut soup
Ingredients: ['chicken', ' curry', ' coconut milk']
Cooking method: Simmered
-------------------------

===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 6
Goodbye.
```

7. Modifikasi dan Refleksi

Perbaikan kecil yang saya lakukan sendiri adalah pemeriksaan judul ganda di awal add_recipe(). Versi pertama fungsi ini langsung menambah resep apa adanya. Masalahnya ketika kami mencoba menambah judul Pancakes untuk kedua kalinya: program sukses, tetapi trie yang menyimpan ID di ujung rantai huruf tertimpa ID baru sehingga hasil pencarian judul itu jadi menunjuk satu resep saja padahal aslinya ada dua. Setelah pemeriksaan ditambahkan, resep berjudul sama tidak pernah diterima lagi dan masalah itu hilang dari akarnya.

Dari proyek ini pelajaran terbesar saya sedikit keluar dari urusan menulis kode: saya belajar menjaga konsistensi data. Sekali saya lupa memanggil rebuild_data() setelah mengubah cook_book, seluruh fitur pencarian teman saya langsung menampilkan data basi, padahal kode mereka tidak punya kesalahan apa-apa. Sejak itu saya selalu menguji perubahan saya lewat opsi pencarian, bukan cuma melihat pesan sukses di fitur sendiri.

Selama mengerjakan proyek ini saya memakai ChatGPT untuk mendiskusikan cara membagi berkas antar anggota dan mengeja ulang konsep linked list saat saya membantu integrasi. Seluruh kode pada controls.py saya tulis dan saya uji sendiri, dan setiap saran dari AI saya verifikasi dulu di program sebelum dipakai, jadi saya bisa menjelaskan semua barisnya.